

How BloodLink works, and how to test all of it

A focused blood-donation platform with four roles and one spine: a recipient asks for blood, an officer finds the nearest eligible compatible donor, the donation is recorded, and stock and histories update. This manual walks each role end to end, maps how the pieces connect, and gives you a role-by-role QA script you can run start to finish.

01 / The cast

Four roles, at a glance

Everyone signs in through one of two doors — the admin portal (`login.php`) or the shared user portal (`user-login.php` , role dropdown). The session `role` gates every page.

SESSION ROLE: ADMIN

Administrator

Runs the bank: manages donors, officers, recipients and blood stock; posts emergency alerts; monitors the request queue; reads reports and the map.

SESSION ROLE: SUPERVISOR

Hospital Officer

The clinical actor: browses the donor pool, works the request queue, matches each request to the nearest eligible donor, and **confirms the donation** — the only role that can.

SESSION ROLE: STUDENT

Blood Donor

Registers with a blood group and address, checks eligibility (the 56-day rule), and sees emergency alerts and nearby blood banks on the map.

SESSION ROLE: RECIPIENT

Recipient

The patient side: registers, files blood requests (urgency-flagged), and tracks each request's status and any donation received.

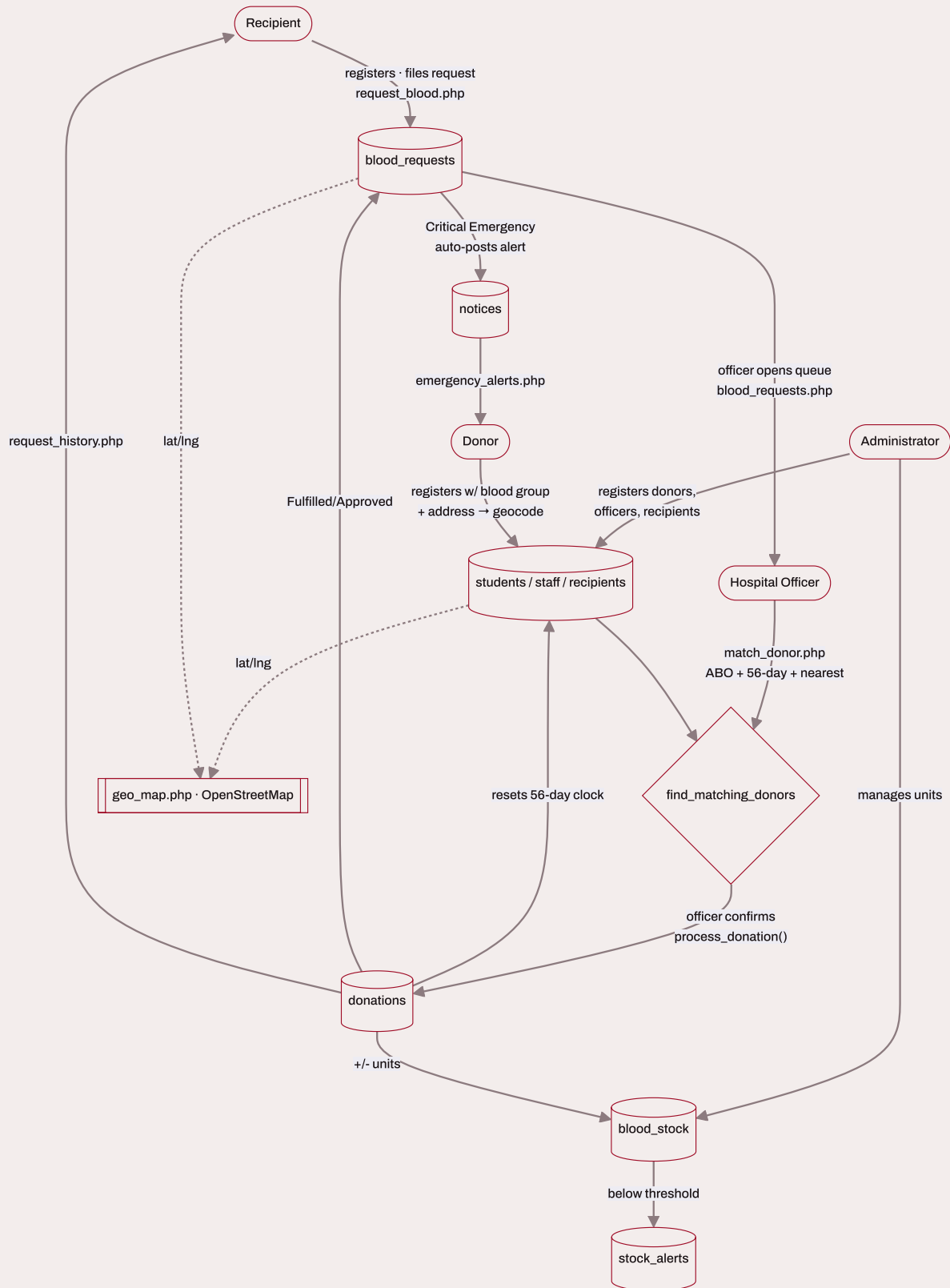
WHY "STUDENT / SUPERVISOR" IN THE CODE?

BloodLink began as a student-supervision (SIWES) project and was converted into a blood bank. The session values and two table names kept the old words — `students` = donors, `staff` = officers — but the student-logbook, attendance, seminar, chat and donor-to-officer "assignment" features have all been removed. It is now purely a blood-donation app; the leftover names are cosmetic.

02 / The spine

How the whole app connects

Everything orbits one workflow: a request becomes a matched, recorded donation. The map and the 56-day rule feed the match; stock and histories fall out of it.



Read it as three inputs feeding one engine:

- **Recipients** create `blood_requests` ; a Critical Emergency also writes a `notices` alert donors can see.
- **Admins** populate the people (`students` = donors, `staff` = officers, `recipients`) and `blood_stock` . There is no donor-to-officer assignment — an officer serves the whole eligible pool.
- **Donors** register into `students` with a blood group and geocoded address.
- **Officers** are the engine: `match_donor.php` runs `find_matching_donors()` (ABO/Rh compatible × 56-day eligible × nearest by distance), and `process_donation()` writes the `donations` row in one transaction that also resets the donor's clock, moves `blood_stock` , updates the request status, and clears any `stock_alerts` .

03 / Data model

Nine tables, and who touches them

TABLE	HOLDS	WRITTEN BY	READ BY
login	Admin accounts	seed	admin login
staff	Hospital officers	admin (add/manage officer)	officer login, matching, reports
students	Donors (+ blood_group, last_donation_date, lat/lng)	donor reg, admin add donor	matching, map, directory, reports
recipients	Patients (+ bcrypt password, lat/lng)	recipient reg, admin add recipient	recipient login, requests
blood_requests	Requests (+ urgency, status, lat/lng)	recipient (request_blood)	officer queue, match, map, history
donations	The donation ledger	officer (process_donation)	histories, reports
blood_stock	Units per blood type	admin, process_donation	stock page, dashboard, reports
stock_alerts	Low-stock alerts	stock page, process_donation	dashboard, stock page
notices	Emergency alerts	officer/admin, auto (critical request)	donor emergency alerts

That is the whole schema. The tables the SIWES original carried — `projects` (assignments), `logbook_entries`, `weekly_summaries`, `attendance`, `seminar`, and `chat_messages` — have been dropped, and the code that touched them removed.

04 / Role walkthroughs

What each role does, end to end

ADMINISTRATOR runs the bank

Signs in at `login.php` (`admin` / `admin`) → `dashboard.php` , which shows live counts (donors, officers, pending requests, low-stock alerts) and recent requests.

- **People:** Manage Donors, Hospital Officers, Recipients — full add / edit / delete (officer & recipient passwords are hashed on create).
- **Blood Stock:** set units + low-stock threshold per type; dropping below threshold raises an alert (and shows on the dashboard), restocking resolves it.
- **Emergency Alerts:** post alerts donors will see.
- **Oversight:** the Blood Requests queue (read-only for admin — officers match), the Geo-Map, and Reports (donations, blood-group spread, top donors, top officers by donations processed, CSV export).

HOSPITAL OFFICER makes the match

Signs in at `user-login.php` (role Hospital Officer, email + password) → `officer_dashboard.php` .

- **Donors:** the donor directory — every donor with blood group and current 56-day eligibility.
- **Blood Requests:** the open queue, most urgent first. "Match Donor" opens `match_donor.php` .
- **Match & confirm:** the match screen lists only ABO/Rh-compatible, 56-day-eligible donors, **nearest first** (distance in km from the request's hospital). Confirming runs `process_donation()` — the one transaction that writes the donation, resets the donor's clock, moves stock, and updates the request.
- **Emergency Alerts & Geo-Map:** post alerts; see donors and requests on the map.

BLOOD DONOR is the supply

Registers at `register.php` (blood group + address, which is geocoded), signs in at `user-login.php` (role Blood Donor, Donor ID + password) →

donor_dashboard.php .

- **Donate Blood:** donation_form.php shows eligibility — whether the donor is inside the 56-day window. Donors do *not* self-record donations; an officer does that on match.
- **Emergency Alerts & Nearby Banks:** see urgent needs and the map.

RECIPIENT creates the demand

Registers at register.php , signs in at user-login.php (role Recipient, email + password) → recipient_dashboard.php .

- **Request Blood:** request_blood.php — patient, group, units, hospital, urgency. A Critical Emergency also auto-posts a donor-facing alert. One pending request per group is enforced; the hospital location is geocoded onto the map.
- **Request History:** every request with its live status, plus any donation received.
- **Nearby Banks:** the map.

05 / The end-to-end donation

Follow one unit of blood

This is the path that ties all four roles together. If this works, the system works.

1

Recipient files a Critical O- request

request_blood.php → a blood_requests row (Pending) + an auto notices alert. Hospital address geocoded.

2

Donors see the alert

Any donor's Emergency Alerts page shows it.

3

Officer opens the queue

blood_requests.php lists it top (Critical first). Officer clicks Match Donor.

4

System proposes donors

`find_matching_donors()` returns only O- donors, eligible under the 56-day rule, ordered by distance to the hospital.

5

Officer confirms

`process_donation()` in one transaction: writes `donations` (Completed), sets donor `last_donation_date` =today, adjusts `blood_stock` , flips the request to Fulfilled/Approved, resolves any `stock_alerts` .

6

Everyone sees the result

Recipient's Request History shows Fulfilled + the donation; the donor is now inside a fresh 56-day window (Donate Blood says "not yet eligible"); admin Reports/Dashboard counts update.

06 / QA runbook

How to test the whole app

Set up once, then run each role's script. The seed is built for this — donors sit on the 56-day boundary and cover all eight blood groups on purpose.

SETUP

Local by Flywheel: start the site, then in its shell:

```
mysql -u root -proot -e "CREATE DATABASE IF NOT EXISTS donor_app"
mysql -u root -proot donor_app < db/schema.sql
mysql -u root -proot donor_app < db/seed.sql
# app at http://<site>.local/bloodlink/ (or php -S localhost:8000 -t .)
# re-run the seed line any time to reset to a known state
```

ROLE	PAGE	USERNAME	PASSWORD
Admin	login.php	admin	admin
Officer	user-login.php	maryam@bloodlink.test	Officer@123
Donor	user-login.php	BL-KD-001	Donor@123
Recipient	user-login.php	hassan@bloodlink.test	Recipient@123

A. AUTHENTICATION & ACCESS (ALL ROLES)

- ✓ Each of the 4 logins lands on the right dashboard. admin→dashboard, officer→officer_dashboard, donor→donor_dashboard, recipient→recipient_dashboard
- ✓ Wrong password is rejected for every role.
- ✓ Logged out, a protected page redirects to login. 302, not a half-rendered page
- ✓ A donor cannot open `dashboard.php` or `officer_dashboard.php` by typing the URL. bounced
- ✓ Logout works from the **topbar avatar dropdown** and the **sidebar** — for every role.
- ✓ Security (optional): `nobody'` OR `'1'='1` as a donor password is rejected; `%` as the admin password is rejected.

B. ADMINISTRATOR

- ✓ Dashboard counts match the data (9 donors, 2 officers, pending requests, 2 low-stock alerts on a fresh seed).
- ✓ Manage Donors shows real blood types (not "N/A"); add a donor, edit it, delete it.
- ✓ Hospital Officers + Recipients: add / edit / delete each; a new officer and recipient can then log in.
- ✓ Blood Stock: drop A+ below threshold → alert appears + dashboard count rises; save again → no duplicate; restock → alert resolves.
- ✓ Reports render; blood-group distribution is populated; Total Donations equals the donations count; CSV export downloads.
- ✓ Geo-Map shows donor + request markers on OpenStreetMap with a visible attribution.

1

Recipient files a request

As Hassan, request O-, Critical Emergency.

- ✓ Success message + it appears in Request History as Pending.
- ✓ A matching alert now exists (check the donor's Emergency Alerts).
- ✓ Re-submitting the same group is refused (one pending per group); units 0 / blank hospital are rejected.

2

Officer matches it

As Maryam, open Blood Requests → Match Donor on that request.

- ✓ Only O- donors appear; an AB+ request would show all groups.
- ✓ Donors are ordered nearest-first with a distance in km.
- ✓ A donor last donated 30 days ago must NOT appear; 56+ days must.

3

Officer confirms the donation

- ✓ Request flips to Fulfilled (units ≥ needed) or Approved.
- ✓ The donor's Donate Blood page now says "not yet eligible" with days remaining.
- ✓ Recipient's Request History shows the donation received.
- ✓ Re-confirming the same donor immediately is refused ("not yet eligible").

D. DONOR & DIRECTORY

- ✓ Register a new donor with an address; log in; the donor appears on the Geo-Map (allow ~1s — Nominatim geocoding).
- ✓ Donate Blood shows the correct eligibility state.
- ✓ As an officer, the Donors directory lists every donor with blood group and an Eligible / "N days to go" badge.

E. CROSS-CUTTING UI/UX

- ✓ Every page: content is clear of the fixed topbar/sidebar (no overlap), tables scroll rather than stretch.
- ✓ The chrome is consistent across all logged-in pages (one white sidebar + topbar), for all four roles.
- ✓ Names and accented text render correctly (e.g. "Officer – Hospital", not mojibake).
- ✓ Resize to a phone width: the sidebar collapses to a hamburger; content goes full-width.
- ✓ Forms reject a tampered/expired session token (CSRF) — a stale form returns an "invalid token" page.

KNOWN LIMITATIONS (EXPECTED, NOT BUGS)

- **Nominatim rate limit (1/sec):** registering several people in quick succession may store NULL coordinates — that person just won't appear on the map. Space them out or re-save.
- **Admins can't confirm donations** by design — the officer is the clinical actor (the donation's officer_id must be a real staff member).
- **Rotate the old Google Maps key** in the Google console — it's out of the code but remains in earlier git history.

BloodLink Operations & QA Manual · roles: admin / supervisor / student / recipient · spine: blood_requests → find_matching_donors → process_donation → donations · map on OpenStreetMap + Nominatim · 9 tables, pure blood-donation app.